

Répondre aux questions suivantes en s'appuyant sur la vidéo accessible [ici](https://dane.web.ac-grenoble.fr/outils-numeriques-0/decoupeur-de-video#VLshgrFke0E) :
<https://dane.web.ac-grenoble.fr/outils-numeriques-0/decoupeur-de-video#VLshgrFke0E>

Comment définir la programmation dynamique ?

.....

.....

Quel type de problème permet de résoudre la programmation dynamique ?

.....

De quand date la programmation dynamique ?

.....

Quel est un inconvénient majeur de la méthode « *diviser pour régner* » ?

.....

.....

.....

.....

En quoi consiste la programmation dynamique ?

.....

.....

Quel est le principe de la méthode ascendante ?

.....

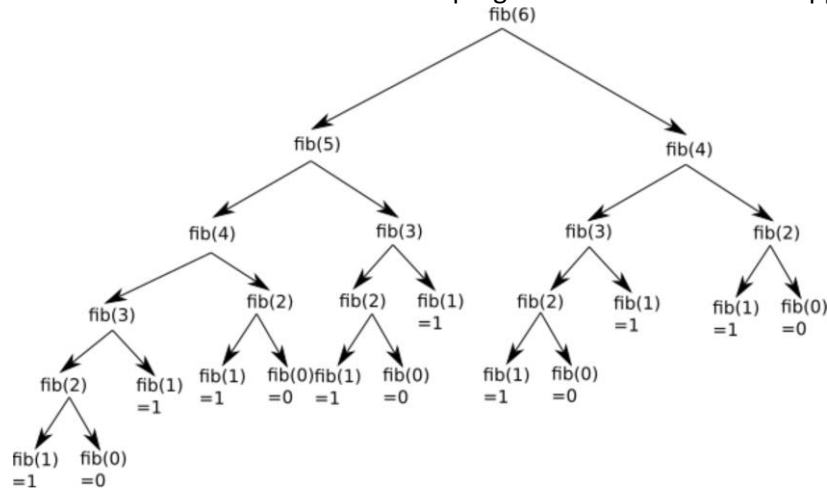
.....

.....

On donne ci-dessous le script d'une fonction récursive permettant de calculer le n -ième terme de la suite de Fibonacci (script déjà rencontré en début d'année, dans le cours sur la récursivité) :

```
1. def fib(n):
2.     if n < 2:
3.         return n
4.     else:
5.         return fib(n - 1) + fib(n - 2)
```

Pour $n = 6$, il est possible d'illustrer le fonctionnement de ce programme avec l'arbre des appels ci-dessous :



En observant attentivement cet arbre, on remarque que plusieurs calculs sont inutiles, car effectués plusieurs fois : par exemple, on retrouve le calcul de `fib(4)` à deux endroits (en haut à droite, et un peu plus bas à gauche) :

On pourrait donc grandement simplifier la résolution du problème `fib(6)` en calculant une fois pour toutes `fib(4)`, en "mémorisant" le résultat et en le réutilisant quand cela est nécessaire. C'est ce que fait le programme suivant.

```

1. def fib_mem(n):
2.     mem = [0]*(n+1) #permet de créer un tableau contenant n+1 zéros
3.     return fib_mem_c(n, mem)
4.
5. def fib_mem_c(n, m):
6.     if (n == 0) or (n == 1):
7.         m[n] = n
8.         return n
9.     elif m[n] > 0:
10.        return m[n]
11.    else:
12.        m[n] = fib_mem_c(n - 1, m) + fib_mem_c(n - 2, m)
13.        return m[n]

```

A faire :

Etudier « à la main » le programme ci-dessus, puis testez-le.

On pourra, dans un second temps, utiliser les sites recursionvisualizer.com et/ou pythontutor.com.

Dans le cas qui nous intéresse, on peut légitimement s'interroger sur le bénéfice de cette opération de "mémoire", mais pour des valeurs de n beaucoup plus élevées, la question ne se pose pas : le gain en termes de performance (temps de calcul) est évident. Pour des valeurs de n très élevées, dans le cas du programme récursif "classique" (n'utilisant pas la "mémoire"), on peut même se retrouver avec un programme qui "plante" à cause du trop grand nombre d'appels récursifs.

En réfléchissant un peu sur le cas que nous venons de traiter, nous divisons un problème "complexe" (calcul de `fib(6)`) en une multitude de petits problèmes faciles à résoudre (`fib(0)` et `fib(1)`), puis nous utilisons les résultats obtenus pour les "petits problèmes" pour résoudre le problème "complexe". Cela devrait vous rappeler la méthode "*diviser pour régner*"... En fait, ce n'est pas tout à fait cela puisque dans le cas de la méthode "*diviser pour régner*", la "mémorisation" des calculs n'est pas prévue.

La méthode que nous venons d'utiliser se nomme "**programmation dynamique**".